

FE-E: Finite-Element and Entropy Control of Adjoint Propagation in Deep Transformers

XUEZHENG WANG

Independent Researcher

Preprint, Version 0.1, August 2026

Abstract

Training very deep Transformers requires task signals to propagate through many residual blocks without developing extreme amplitudes, abrupt layer-to-layer changes, or concentration in a small subset of depths. Existing stabilization methods primarily modify residual parameterization, initialization, normalization, or optimizer updates. We introduce FE-E, a complementary method that treats the task-loss gradients with respect to hidden residual states as a vector-valued adjoint field on a one-dimensional finite-element mesh over network depth. FE-E combines three terms: a stiffness energy that penalizes rapid depth-wise variation, a consistent-mass energy anchor that prevents the trivial zero-gradient solution and controls absolute scale, and a finite-element-relative entropy band that discourages pathological concentration without forcing uniformity. The exact objective is differentiable and therefore requires double backpropagation.

We implement FE-E in PyTorch and compare it directly with AdamW and the recent Gradient Smoothing method using an offline reverse-sequence Transformer benchmark. On a 24-layer, five-seed confirmation study with 200 updates, FE-E reaches an evaluation loss of 3.0103 ± 0.0233 , compared with 3.0911 ± 0.0228 for Gradient Smoothing and 3.1121 ± 0.0232 for AdamW. The paired FE-E versus Gradient Smoothing loss difference is -0.0808 with a 95% t interval of $[-0.1233, -0.0383]$. A 48-layer audit preserves the fixed-step ordering. However, exact FE-E costs 2.29x to 2.64x the baseline step time, and Gradient Smoothing remains better under the approximate compute-matched budgets tested here. These results establish FE-E as an effective per-update control mechanism and depth-wise diagnostic, while also identifying second-order cost as the central obstacle to scale. We release the implementation, tests, raw results, and reproducibility scripts with this preprint.

1. Introduction

The Transformer architecture [1] has become a standard substrate for sequence modeling, but increasing depth remains harder than increasing width under many training configurations. Gradients can vanish or explode, residual perturbations can be amplified, and forward representations can collapse or become poorly conditioned. The placement of layer normalization affects gradient behavior at initialization [2], residual dependence can amplify small parameter updates [3], and carefully designed residual scaling and initialization can enable much deeper models [4-6]. These results show that depth stability is not a single optimizer problem.

Recent Gradient Smoothing [7] adds a new perspective: block-wise optimizer updates exhibit exploitable structure along depth, and locally smoothing those updates can improve optimization with small overhead. FE-E starts from a different object. For a task loss L_{task} and residual states $h_0, \dots, h_L$, we study the hidden-state adjoints

$$g_l = \partial L_{\text{task}} / \partial h_l$$

These adjoints are batch-, token-, and task-conditioned signals that describe how the current objective reaches each depth. Parameter-update smoothness and hidden-adjoint smoothness are related through the network Jacobians, but they are not equivalent. A smooth AdamW update field does not guarantee a smooth or well-scaled hidden adjoint field, and a stable hidden adjoint field need not imply similar parameter updates.

FE-E interprets network depth as a one-dimensional mesh and the adjoints as nodal values of a piecewise-linear vector field. This makes standard finite-element quantities available as explicit diagnostics and regularizers. The stiffness matrix measures depth-wise roughness. The consistent mass matrix measures integrated adjoint energy. A finite-element-relative entropy measures concentration with respect to the depth quadrature measure. Their roles are deliberately separated: stiffness controls shape, mass controls amplitude, and entropy controls distribution.

The method is finite-element-inspired rather than a claim that a standard Transformer solves a physical partial differential equation. For ordinary unit residual blocks, depth is parameterized by layer index. A unit-interval parameterization is consistent only when residual increments also scale as $O(1/L)$, as in a genuine fixed-horizon continuous-depth discretization [10]. This distinction matters because otherwise the stiffness energy changes artificially with depth.

This paper makes four contributions:

- It formulates hidden-state task adjoints as a finite-element field over Transformer depth and derives stiffness, consistent-mass, and relative-entropy measurements.
- It introduces the FE-E objective, including an amplitude anchor and an entropy target band that avoid two degeneracies of naive smoothing: the zero field and forced uniformity.
- It provides an energy-control interpretation showing why mass and stiffness must be controlled jointly, and makes the double-backpropagation requirement explicit.
- It reports a compute-aware comparison with Gradient Smoothing, including positive fixed-step results, negative compute-matched results, stress tests, geometry controls, and fully reproducible code.

The evidence is intentionally scoped. The experiments use a small synthetic Transformer benchmark on CPU. They establish a mechanism and generate falsifiable hypotheses; they do not establish large-language-model superiority.

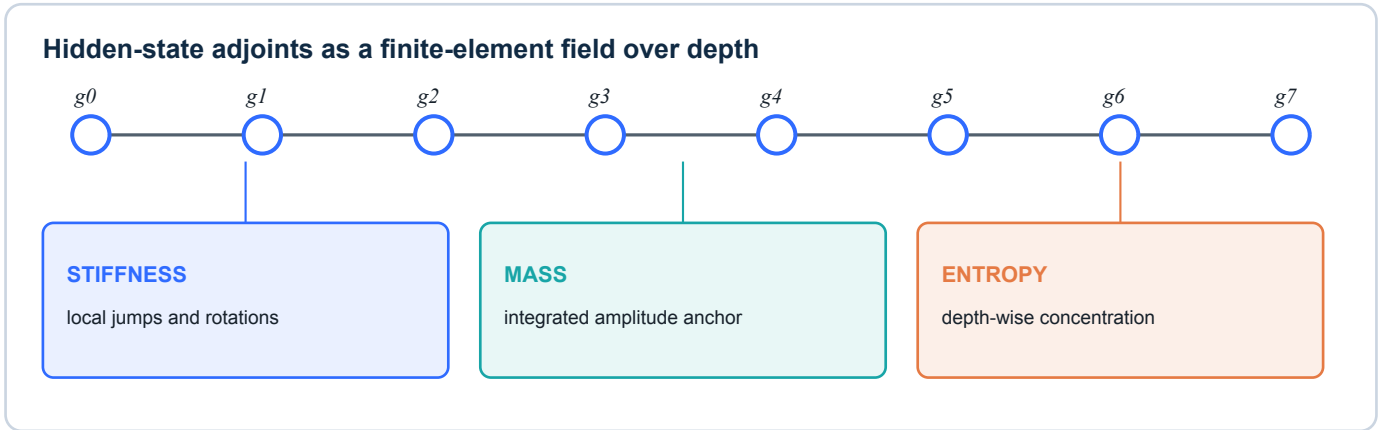


Figure 1. FE-E treats task-loss gradients with respect to residual states as a depth-wise finite-element field. Stiffness controls local shape, mass controls absolute energy, and relative entropy controls concentration.

2. Related Work

2.1 Deep Transformer stabilization

Pre-LN placement improves gradient behavior at initialization and reduces dependence on learning-rate warmup [2]. Admin attributes instability primarily to residual amplification rather than gradient imbalance alone [3]. DeepNorm combines residual scaling and initialization to bound model updates and train Transformers with up to 1,000 layers [4]. DeepScaleLM develops end-to-end moment propagation theory and similarly demonstrates very deep models [5]. ReZero initializes residual gates at zero and obtains initial dynamical isometry [6]. FE-E does not replace these architectural controls. It measures and regularizes the task-conditioned adjoint field produced after architecture, initialization, and data interact.

Gradient clipping [16] controls aggregate parameter-gradient magnitude but does not encode which depth interval generated a jump or where energy is concentrated. FE-E instead defines local and global quantities on the residual-state path.

2.2 Depth-wise update smoothing

Gradient Smoothing [7] transforms block-wise updates generated by a base optimizer. Its Window operator replaces an interior update u_l by $(1-\alpha)u_l + \alpha(u_{l-1} + u_{l+1})/2$, with one-sided boundary weights. It can be applied after AdamW preconditioning and before the parameter update. Our implementation follows that ordering and leaves decoupled weight decay [14] outside the smoothing operator. FE-E differs in three ways: it acts on hidden adjoints, enters the objective rather than post-processing an update, and explicitly includes amplitude and concentration controls.

2.3 Entropy and representation collapse

Attention entropy collapse has been associated with unstable Transformer training [8], but attention entropy is a distribution over tokens inside an attention head. FE-E entropy is a distribution of adjoint energy over depth. The two quantities are not interchangeable. Likewise, the rank-collapse analysis of pure attention [9] concerns forward token representations. FE-E therefore reports forward representation diagnostics separately and does not infer representation rank from gradient entropy.

2.4 Finite elements, continuous depth, and double backpropagation

Linear finite elements provide consistent stiffness and mass bilinear forms for piecewise-linear fields [11]. Neural ordinary differential equations motivate continuous-depth interpretations [10], but FE-E only uses the discrete variational structure and does not require an ODE solver. Because the regularizer depends on gradients of the task loss, differentiating it with respect to parameters is a form of double backpropagation [13]. The released implementation uses PyTorch higher-order automatic differentiation [15].

3. FE-E Method

3.1 Depth mesh and adjoint field

Let a Transformer contain L residual blocks and return residual-stream states $h_0, h_1, \dots, h_L$. Assign strictly increasing coordinates $z_0, \dots, z_L$ and element lengths $\Delta z_l = z_{l+1} - z_l$. For ordinary Transformers we use $z_l = l$. For a fixed-horizon discretization with residual scale $O(1/L)$, $z_l = l/L$ is appropriate.

For task loss L_{task} , define

$$g_l = \partial L_{\text{task}} / \partial h_l, l = 0, \dots, L.$$

Each g_l has the batch, token, and feature dimensions of h_l . Inner products below are averaged over these dimensions, optionally with a padding mask. Stack the nodal adjoints into $G = [g_0, \dots, g_L]^T$.

3.2 Stiffness energy

Linear interpolation on each depth element gives a constant derivative. The discrete stiffness energy is

$$E_{\text{stiff}} = \sum_{l=0}^{L-1} \|\mathbf{g}_{l+1} - \mathbf{g}_l\|^2 / \Delta z_l = G^T (K \square I) G.$$

This term detects amplitude jumps, direction rotations, and high-frequency oscillation. A norm-only proxy, based on differences of $\|\mathbf{g}_l\|$, cannot detect a direction reversal with unchanged norm; our unit tests include this counterexample. The default FE-E implementation therefore uses full vector differences.

Raw stiffness scales with adjoint magnitude. For the trainable penalty we use

$$R_{\text{stiff}} = E_{\text{stiff}} / (E_{\text{mass}} + \varepsilon),$$

which measures relative roughness while the separate mass term retains amplitude information.

3.3 Consistent mass energy

For the same piecewise-linear field, the local mass contribution on element l is

$$E_{\text{mass},l} = (\Delta z_l / 3) [\|\mathbf{g}_l\|^2 + \langle \mathbf{g}_l, \mathbf{g}_{l+1} \rangle + \|\mathbf{g}_{l+1}\|^2].$$

Summing elements yields

$$E_{\text{mass}} = G^T (M \square I) G.$$

A stiffness-only objective admits the zero field as an easy optimum. FE-E anchors mass energy to a reference E_{ref} using a symmetric logarithmic ratio:

$$R_{\text{mass}} = [\log((E_{\text{mass}} + \varepsilon) / (E_{\text{ref}} + \varepsilon))]^2.$$

The reference can be fixed at the initial observed energy, supplied externally, or updated by a slow exponential moving average. The experiments use the fixed-initial reference.

3.4 Finite-element-relative entropy

Let w_l be the lumped finite-element quadrature weights: each interior node receives half of each adjacent element and boundary nodes receive half of their single adjacent element. Define node energy $e_l = \|g_l\|^2$ and

$$p_l = w_l e_l / \sum_j w_j e_j, \quad r_l = w_l / \sum_j w_j.$$

Instead of normalizing Shannon entropy only by $\log(L+1)$, FE-E measures divergence from the mesh measure:

$$D_{depth} = \sum_l p_l \log(p_l / r_l), \quad C_{depth} = \exp(-D_{depth}).$$

The coverage C_{depth} lies in $(0, 1]$ and equals one for constant energy density, including on nonuniform meshes. Low coverage indicates concentration in a small depth region. Entropy is scale-free, so it cannot detect a uniform 1,000-fold explosion or collapse; this is why the mass anchor is essential.

Maximizing coverage unconditionally could erase useful depth specialization. FE-E instead uses a band $[C_{min}, C_{max}]$:

$$R_{entropy} = [C_{min} - C_{depth}]_-^2 + [C_{depth} - C_{max}]_-^2.$$

3.5 Total objective and higher-order differentiation

The complete objective is

$$L_{total} = L_{task} + s(t)[\lambda_s R_{stiff} + \lambda_m R_{mass} + \lambda_h R_{entropy}],$$

where $s(t)$ is a warmup multiplier. Because each g_l is itself a derivative, the parameter gradient of the FE-E penalty contains partial g_l / partial theta. Exact optimization therefore requires a retained higher-order graph. Detached hooks can compute FE-E diagnostics but cannot train the regularizer.

3.6 Energy-control interpretation

Let $g_h(z)$ be the piecewise-linear interpolant on an interval of length T . The finite-element matrices exactly represent its L_2 and H_1 -seminorm energies:

$$\|g_h\|_{L_2}^2 = E_{mass}, \quad \|\partial_z g_h\|_{L_2}^2 = E_{stiff}$$

For any one-dimensional H_1 field, a direct fundamental-theorem and Cauchy-Schwarz argument gives

$$\|g_h\|_{\infty} \leq T^{1/2} E_{mass}^{1/2} + T^{1/2} E_{stiff}^{1/2}.$$

Thus anchoring mass while controlling relative stiffness bounds depth-local peaks in the interpolated adjoint. This statement explains propagation shape control; it is not a convergence or generalization guarantee for nonconvex training.

4. Experimental Protocol

4.1 Offline task and model

All experiments are generated offline. Inputs are length-12 sequences sampled uniformly from a vocabulary of 32 symbols. The target is the reversed input sequence. The model is a Pre-LN Transformer with four attention heads, width 32, MLP expansion two, learned positional embeddings, and unit residual scale unless stated otherwise. The 24-layer and 48-layer models contain 207,552 and 412,608 trainable parameters, respectively.

Training uses AdamW with learning rate 0.002, weight decay 0.01, batch size 8, and no gradient clipping. Evaluation averages ten deterministic batches that are disjoint by pseudorandom index from the training stream. All compared variants in a seed start from the same initialization and see the same training and evaluation sequences.

4.2 Methods and hyperparameters

The main comparison contains:

- AdamW baseline, implemented as the $\alpha = 0$ case of the same optimizer wrapper used by Gradient Smoothing. Multiple-step tests match PyTorch AdamW numerically.
- Gradient Smoothing, Window Standard/Proj, applied to linear projection parameters after AdamW preconditioning. Development seed 7 evaluates α in $\{0.05, 0.10, 0.20\}$; $\alpha = 0.20$ is selected before the confirmation seeds.
- FE-E with $\lambda_s = 2$, $\lambda_m = 0.02$, $\lambda_h = 2$, coverage band $[0.90, 0.98]$, fixed-initial mass reference, and ten-step linear warmup. These coefficients are inherited from the preceding mechanism study and are not retuned on the confirmation seeds.

The 24-layer confirmation uses seeds 31, 47, 59, 71, and 89 for 200 updates, with evaluation every 25 updates and diagnostics every 20 updates. The 48-layer audit uses seeds 31, 47, and 59 for 100 updates.

4.3 Metrics and statistical reporting

We report evaluation cross-entropy, token accuracy, mean observed CPU step time, normalized adjoint stiffness, mass energy, relative coverage, optimizer-update roughness, and forward residual-direction diagnostics. Five-seed paired differences use a two-sided 95% t interval with four degrees of freedom. The shared-CPU wall-clock measurements are noisy; within-seed ratios and a backward-count proxy are reported together.

Approximate compute matching gives exact FE-E 100 steps for every 200 first-order steps. This is generous to FE-E because measured exact costs exceed 2x in most comparisons. A second wall-clock view uses each seed's AdamW runtime as a budget and selects the latest available evaluation checkpoint within that budget.

5. Results

5.1 Component ablation

Table 1 aggregates three seeds after 40 updates. Stiffness alone strongly reduces roughness but collapses mass energy and worsens evaluation loss. The mass term prevents this zero-field tendency. Entropy alone changes little because the baseline is not severely concentrated. The full combination produces the most balanced internal diagnostics, but the mass-only variant has the best task loss in this short ablation. This result rejects the claim that every FE-E component independently improves prediction.

Table 1. Three-seed component ablation at 24 layers and 40 updates.

Variant	Eval. loss	Accuracy	Rel. stiffness	Mass energy	Coverage	Time/base
AdamW	3.3702 0.0321	0.060 ± 0.011	0.0923	1.00e-6	0.796	1.00x
Stiffness	3.4666 0.0021	0.035 ± 0.007	0.0068	0.17e-6	0.999	2.19x
Mass	3.3447 0.0254	0.079 ± 0.007	0.0745	3.76e-6	0.800	2.32x
Entropy	3.3857 0.0203	0.058 ± 0.010	0.0915	0.99e-6	0.805	2.09x
FE-E	3.3572 0.0236	0.072 ± 0.012	0.0264	4.21e-6	0.950	2.36x

5.2 Direct comparison at fixed update count

Table 2 gives the primary five-seed result. Gradient Smoothing improves evaluation loss over AdamW by 0.0209 on average, with 95% interval $[-0.0264, -0.0154]$. FE-E improves over AdamW by 0.1018, interval $[-0.1418, -0.0617]$. The paired FE-E minus Gradient Smoothing difference is -0.0808 , interval $[-0.1233, -0.0383]$. All five loss differences favor FE-E. FE-E accuracy is higher on average, but the FE-E versus Gradient Smoothing accuracy interval slightly crosses zero and is not treated as confirmed.

Table 2. Five-seed direct comparison at 24 layers and 200 updates.

Method	Eval. loss	Accuracy	Time/base	Rel. stiffness	Coverage
AdamW	3.1121 ± 0.0232	0.125 ± 0.010	1.00x	0.1158	0.705
Gradient Smoothing	3.0911 ± 0.0228	0.129 ± 0.010	1.26x	0.0968	0.756
FE-E	3.0103 ± 0.0233	0.144 ± 0.007	2.29x	0.0168	0.938

The result demonstrates higher per-update effectiveness, not higher compute efficiency. FE-E directly regularizes the task adjoints and is permitted more automatic-differentiation work per update.

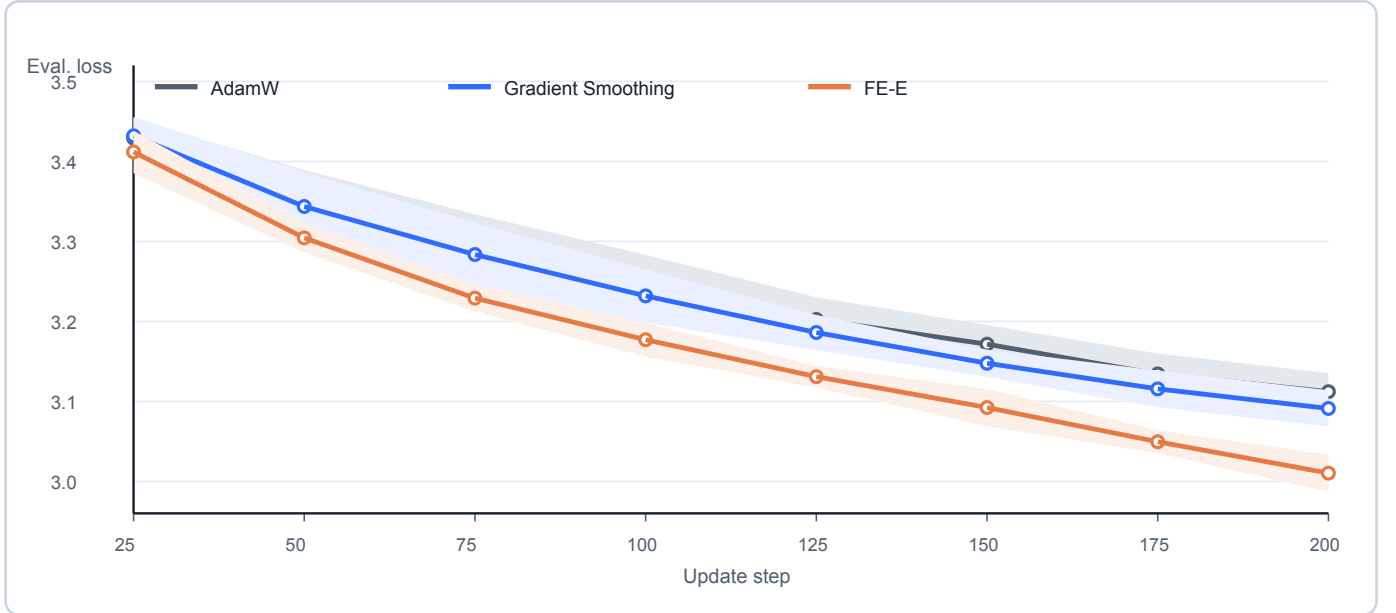


Figure 2. Mean evaluation loss over five confirmation seeds; shaded regions show one sample standard deviation. FE-E has the best fixed-step curve in this short synthetic experiment.

5.3 Compute-budget comparison

Under the approximate backward-count budget, AdamW and Gradient Smoothing retain 200 updates while FE-E receives 100. Gradient Smoothing has the lowest loss. The stricter observed baseline-wall-clock budget yields the same ordering: Gradient Smoothing averages 160 completed updates and loss 3.1364, whereas FE-E averages 80 updates and loss 3.2273.

Table 3. Compute-aware comparison. Lower loss is better.

Method	Approx. backward-budget loss	Baseline-wall-clock loss	Avg. steps in wall-clock budget
AdamW	3.1121 ± 0.0232	3.1121 ± 0.0232	200
Gradient Smoothing	3.0911 ± 0.0228	3.1364 ± 0.0391	160
FE-E	3.1770 ± 0.0213	3.2273 ± 0.0629	80

This negative result is central: exact FE-E presently exchanges throughput for stronger control, and abundant hardware does not by itself establish a compute-matched advantage because optimization steps remain sequential.

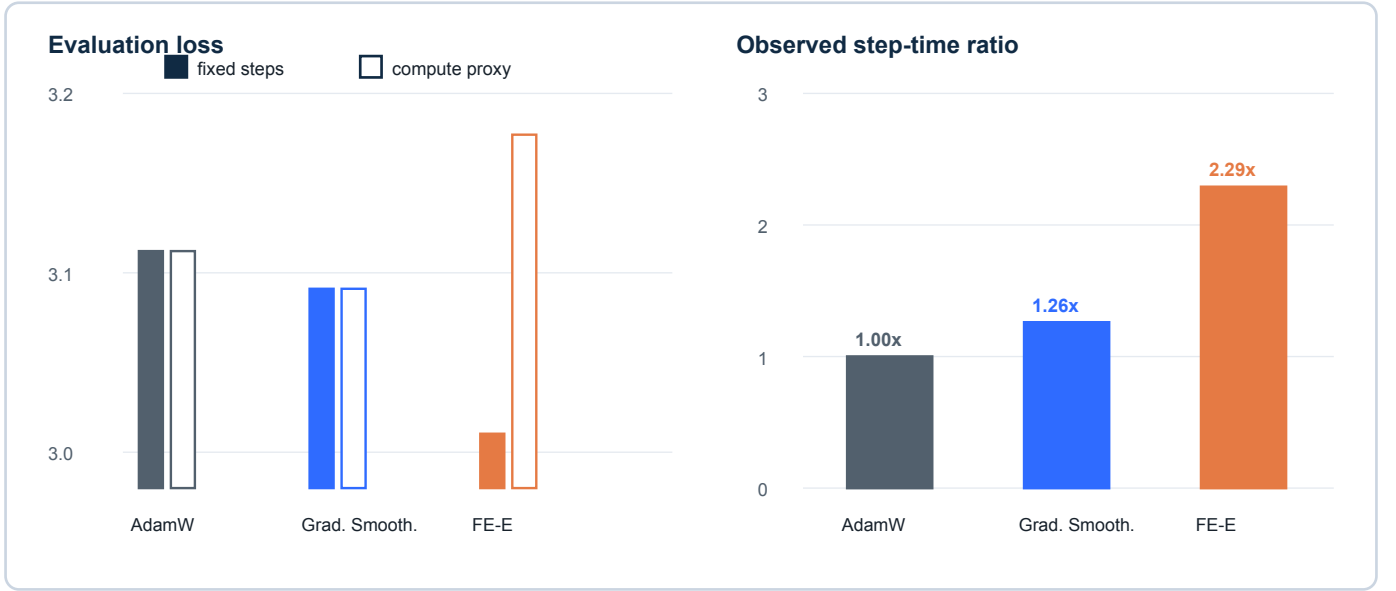


Figure 3. Fixed-step quality and compute efficiency separate. Filled loss bars use equal update counts; outlined bars use the approximate backward-count budget. Exact FE-E has the strongest per-update result but the highest observed cost.

5.4 Forty-eight-layer depth audit

The 48-layer, three-seed audit uses the same hyperparameters without retuning. At 100 updates, FE-E again has the lowest mean loss and all per-seed loss differences have the same sign. Its observed step cost increases to 2.64x baseline. With FE-E reduced to 50 steps as an approximate backward-count match, its loss becomes 3.3514 ± 0.0134 , worse than Gradient Smoothing at 3.2912 ± 0.0457 .

Table 4. Three-seed descriptive depth audit at 48 layers.

Method	Eval. loss at 100 steps	Accuracy	Time/base
AdamW	3.3068 ± 0.0415	0.080 ± 0.014	1.00x
Gradient Smoothing	3.2912 ± 0.0457	0.089 ± 0.019	1.28x
FE-E	3.2394 ± 0.0349	0.093 ± 0.011	2.64x

The audit supports depth robustness of the fixed-step ordering but has only three seeds and is not an independent significance claim.

5.5 Mechanism measurements and negative representation result

At the common diagnostic checkpoint, Gradient Smoothing reduces the roughness of applied projection updates by 26.1% and raises their adjacent cosine from 0.027 to 0.283. FE-E instead reduces normalized hidden-adjoint stiffness to 0.0168, approximately one sixth of the Gradient Smoothing value, while maintaining higher mass energy and coverage. The methods therefore modify their intended, different objects.

The forward residual-direction diagnostic does not improve. Mean adjacent residual cosine is 0.365 for AdamW, 0.297 for Gradient Smoothing, and 0.242 for FE-E. The reverse-sequence experiment therefore does not reproduce the improved representation-path alignment reported for Gradient Smoothing on vision models [7]. This measurement prevents interpreting adjoint smoothness as forward-representation smoothness.

5.6 Stress test and geometry control

A 48-layer stress configuration uses width 24, sequence length 8, residual scale 3, learning rate 0.005, and 30 updates. AdamW remains finite. FE-E reduces normalized stiffness from approximately 0.261 to approximately 0.01 and raises coverage from 0.625 to approximately 0.99, but evaluation loss worsens from 3.456 to 3.575. Without warmup, the first-stage FE-E parameter-gradient peak reaches 11.30. A ten-step warmup reduces the peak to 2.07 but does not recover task performance. Better internal diagnostics are therefore not sufficient evidence of better learning.

A separate 24-layer control sets $z_1 = 1/L$ and residual scale $1/L$. Baseline coverage is already near one, and FE-E slightly worsens evaluation loss from 3.518 to 3.536. A fixed entropy target is not architecture-invariant; it must be calibrated to the baseline geometry and residual parameterization.

6. Engineering Interpretation

FE-E converts depth stability from an indirectly observed property into a measurable state with three coordinates: roughness, amplitude, and concentration. This suggests three deployment levels.

First, diagnostic FE-E computes the quantities with `create_graph` disabled and does not change training. It can compare initialization schemes, localize unstable depth intervals, and trigger alerts. Second, sparse-control FE-E activates the regularizer every k steps or only after a threshold crossing. Third, full FE-E is a research-grade upper bound for settings in which training failure is more expensive than additional compute.

The most plausible production architecture is hybrid: Gradient Smoothing runs continuously as a low-cost update transformation, FE-E diagnostics run periodically on the task-conditioned hidden adjoints, and exact FE-E activates transiently when stiffness, mass, or coverage leaves a calibrated region. This architecture treats FE-E as a control plane rather than as a universal optimizer replacement.

Integration with large-scale training is nontrivial. Residual states must remain available; exact regularization prolongs the higher-order graph lifetime; activation checkpointing and fused attention must support second derivatives; small mass energies and logarithmic ratios should accumulate in FP32 under mixed precision; and FSDP or pipeline parallelism must aggregate or partition the depth mesh consistently. Peak GPU memory may become limiting before arithmetic throughput.

Engineering acceptance must use time-to-target loss, total GPU hours, peak memory, tokens per second, loss-spike rate, nonfinite-run rate, and retry cost. Fixed update count alone favors methods that spend more work per step.

7. Limitations and Research Agenda

The present study is deliberately small. It uses synthetic data, short optimization horizons, CPU timings, and no GPU memory measurement. It has no autoregressive language-model benchmark, no full Jacobian spectrum, and no comparison yet with DeepNorm, DeepScaleLM, ReZero, or clipping under one unified codebase. Hyperparameters receive only a limited mechanism-driven calibration. The 48-layer audit has three seeds.

The highest-priority question is whether an efficient FE-E approximation preserves most of the fixed-step benefit. Candidate approximations include periodic activation, adaptive threshold activation, depth-node subsampling, block grouping, norm-only screening followed by vector confirmation, and randomized projections of the adjoint vector. A publishable large-scale result should show at least one of the following under equal GPU hours: lower validation loss, fewer instability events, a deeper trainable model, or lower expected compute after accounting for failed-run retries.

The entropy term also requires continued scrutiny. Coverage measures depth concentration of adjoint energy, not semantic information, mutual information, forward representation rank, or attention entropy. If FE-E only improves its own diagnostic values without improving task quality or stability events, the appropriate outcome is a propagation diagnostic tool rather than a training algorithm.

8. Conclusion

FE-E introduces finite-element stiffness, consistent-mass energy, and mesh-relative entropy as complementary controls on hidden-state task adjoints in deep Transformers. Small controlled experiments show a statistically consistent fixed-step loss advantage over AdamW and Gradient Smoothing and a much smoother hidden-adjoint field. They also show that exact double backpropagation currently loses under compute matching. FE-E is therefore best understood as a promising adjoint-propagation control framework whose scientific opportunity lies in depth scaling and whose engineering challenge is efficient, event-driven approximation.

Appendix A. Implementation and Reproducibility Details

The released package exposes `FEERegularizer` and retains `FEEntropyRegularizer` as a compatibility alias. The experimental CLI uses `fe_e`, while old JSON records use the legacy variant label `fe_entropy`. The Gradient Smoothing implementation supports Standard, Norm-Preserving, and Directional variants with Proj or Full scope. Alpha zero matches PyTorch AdamW over multiple steps. Tests cover finite-element analytic fields, mesh refinement, padding masks, direction-reversal detection, entropy scale invariance, active double backpropagation, detached-regularizer rejection, exact window boundaries, norm preservation, and smoothing roughness.

The attachment contains:

- `src/fe_entropy/`: FE-E, Gradient Smoothing AdamW, model, spectral diagnostics, and experiment CLI.
- `tests/`: fifteen unit and automatic-differentiation tests.
- `results/`: raw JSON for confirmation, depth audit, ablation, stress, and geometry-control runs.
- `scripts/`: aggregation and reproducibility helpers.
- `paper/`: editable preprint source and PDF builder.

The verified test command is:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
```

The exact environment used for the reported runs is PyTorch 2.13.0+cpu inside the recorded container workflow. Data are generated deterministically and require no external download.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems* 30, 2017. <https://arxiv.org/abs/1706.03762>
- [2] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu. On Layer Normalization in the Transformer Architecture. *Proceedings of ICML*, PMLR 119:10524-10533, 2020. <https://proceedings.mlr.press/v119/xiong20b.html>
- [3] L. Liu, X. Liu, J. Gao, W. Chen, and J. Han. Understanding the Difficulty of Training Transformers. *Proceedings of EMNLP*, pages 5747-5763, 2020. <https://aclanthology.org/2020.emnlp-main.463/>
- [4] H. Wang, S. Ma, L. Dong, S. Huang, D. Zhang, and F. Wei. DeepNet: Scaling Transformers to 1,000 Layers. *arXiv:2203.00555*, 2022. <https://arxiv.org/abs/2203.00555>
- [5] A. Kedia, M. A. Zaidi, S. Khyalia, J. Jung, H. Goka, and H. Lee. Transformers Get Stable: An End-to-End Signal Propagation Theory for Language Models. *Proceedings of ICML*, PMLR 235:23449-23531, 2024. <https://proceedings.mlr.press/v235/kedia24a.html>
- [6] T. Bachlechner, B. P. Majumder, H. Mao, G. Cottrell, and J. McAuley. ReZero is All You Need: Fast Convergence at Large Depth. *Proceedings of UAI*, PMLR 161:1352-1361, 2021. <https://proceedings.mlr.press/v161/bachlechner21a.html>
- [7] H. Meng, A. Sugolov, and V. Papayan. Gradient Smoothing: Coupling Layer-wise Updates for Improved Optimization. *arXiv:2606.30813*, 2026. <https://arxiv.org/abs/2606.30813>
- [8] S. Zhai, T. Likhomanenko, E. Littwin, D. Busbridge, J. Ramapuram, Y. Zhang, J. Gu, and J. M. Susskind. Stabilizing Transformer Training by Preventing Attention Entropy Collapse. *Proceedings of ICML*, PMLR 202:40770-40803, 2023. <https://proceedings.mlr.press/v202/zhai23a.html>
- [9] Y. Dong, J.-B. Cordonnier, and A. Loukas. Attention Is Not All You Need: Pure Attention Loses Rank Doubly Exponentially with Depth. *Proceedings of ICML*, PMLR 139:2793-2803, 2021. <https://proceedings.mlr.press/v139/dong21a.html>
- [10] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural Ordinary Differential Equations. *Advances in Neural Information Processing Systems* 31, 2018. <https://proceedings.neurips.cc/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html>
- [11] P. G. Ciarlet. *The Finite Element Method for Elliptic Problems*. North-Holland, 1978.
- [12] C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27:379-423 and 623-656, 1948. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- [13] H. Drucker and Y. LeCun. Improving Generalization Performance Using Double Backpropagation. *IEEE Transactions on Neural Networks*, 3(6):991-997, 1992.
- [14] I. Loshchilov and F. Hutter. Decoupled Weight Decay Regularization. *International Conference on Learning Representations*, 2019. <https://arxiv.org/abs/1711.05101>
- [15] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems* 32, 2019. <https://pytorch.org/assets/pytorch2-2.pdf>
- [16] R. Pascanu, T. Mikolov, and Y. Bengio. On the Difficulty of Training Recurrent Neural Networks. *Proceedings of ICML*, PMLR 28(3):1310-1318, 2013. <https://proceedings.mlr.press/v28/pascanu13.html>